Frontend System Design Doc

Supertalent Travel OTA — Page 1 (Header, Navigation, and Landing Page)

Author: Addy Hebou | *Status: Draft — pre-engagement scoping* | *Last updated: Aug 12, 2026*

This doc scopes the frontend architecture for the first shippable slice of the Supertalent Travel OTA: the global header/nav, hero section, and navigation. It follows a RADIO structure: Requirements, Architecture, Data and State Layer, Interface (API contracts), and Optimizations.

1. Requirements

1.1 Functional Requirements

- Sticky global header with 3-column layout: search (left), logo (center), account/cart (right); background transitions smoothly on scroll.
- Search: collapsed by default (icon + label only); on click, expands inline or opens a full-screen overlay with smooth fade-in.
- Global nav: 7 category menu items (Global Chains, Free Travel, Exhibitions, Honeymoon, Golf, Castle, Villas).
- Hero section: full-bleed image/video background, tagline copy, "Start the Journey" CTA that smooth-scrolls or transitions to booking flow.

1.2 Non-Functional Requirements

- Perceived interaction latency under 100ms for cached brand/city data (no visible loading state on repeat interaction).
- Zero full-page reloads or flicker during any navigation or menu interaction (true SPA behavior).
- Responsive from 375px (mobile) to 1280px+ (desktop); mega-menu degrades to an accordion/drawer pattern on mobile.
- Visual standard: generous negative space, restrained type scale, no default/browser-native form controls (date picker is fully custom).
- Images lazy-loaded from CDN and served at appropriate sizes per viewport (no full-res hero images on mobile).
- Accessible: keyboard-navigable mega-menu and date picker, disabled dates announced to screen readers.

2. Architecture — Component Tree

Page 1 is composed of a persistent shell (header/footer) wrapping route-level content. Global Chains is treated as a nested route so its state (active brand, selected city/dates) is URL-shareable and doesn't require prop-drilling from the shell.

```
<AppShell>
 <Header>
  <AppNav>
    <SearchTrigger />      // collapsed -> SearchOverlay
    <Logo />
    <AccountCartMenu />
  <AppNav>
```

```
 <PrimaryNav>          // 7 category links
   <NavItem onHover={openMegaMenu} />
 </PrimaryNav>
</Header>



<MegaMenu category={hoveredNavBarItem} />   // portal, renders over content


I.e example for hoveredNavBarItem: string = 'global-chains'
 <MegaMenu category="global-chains" />   // portal, renders over content
  <BrandList />          // 110+ brands, virtualized list
  <CityPanel brandId />   // driven by selected brand
  <DatePickerPanel />     // appears once brand+city context exists
  <SearchCTA />
 </MegaMenu>

 <main>
  <HeroSection />        // route: "/"
  <!-- future categories mount here as routes -->
 </main>

 <Footer />
</AppShell>
```

2.1 Component Notes

Component	Responsibility
Header	Layout shell only; owns scroll-position state for sticky background transition.
SearchTrigger / SearchOverlay	Local UI state (open/closed). No server data.
MegaMenu	Rendered via portal so it can overlay page content without layout shift; owns "active brand" UI state.
BrandList	Renders from a React Query cache of /brands; virtualized (react-window) given 110+ items.
CityPanel	Subscribes to /brands/:id/cities via React Query, keyed by active brand id.
DatePickerPanel	Fully custom (no native <input type="date">); disabled-date logic is pure/local, no fetch.
SearchCTA	Derived/enabled state only when brand + city + dates are all set; triggers route push to results page.

3. Data Layer

All server-derived data (brands, cities, availability) is fetched and cached via React Query — not component state — so that repeat interactions (re-hovering a brand you already viewed) are instant and don't re-fetch. This also gives us request de-duplication, background refetch, and stale-time control for free.

- Query keys are hierarchical: ['brands'], ['brands', brandId, 'cities'] — enables targeted cache invalidation later without over-fetching.
- staleTime set high (e.g. 10 min) for brand/city data — this is near-static reference data, not live inventory.
- Mutations are out of scope for Page 1 (no write operations until the actual booking/checkout flow).
- Local/UI-only state (active menu item, search overlay open, hover state) stays in component state via useState/useReducer — it is not server data and does not belong in the query cache.

3.1 Mock Data Strategy (pre-backend-integration)

Since Phase 1 development starts before live API access is confirmed, the data layer is built against a mock service (MSW — Mock Service Worker) that intercepts requests at the network level. This means React Query, loading states, and error handling are all exercised realistically — and swapping the mock layer for the real backend later is a one-line change to the base URL, not a refactor.

State Layer	**Values**	**Subscribing Components**	**Notes**
useState	isScrolled (header bg transition) isSearchOpen: boolean – collapsed/expanded activeCategory: Category – gnb item hovered – activeBrandID: string – selectedCity: string – checkIn: number – checkOut: number	<Header /> <SearchTrigger /><SearchOverlay/> <AppShell /> props down activeCategory to <PrimaryNav /> and <MegaMenu /> <GlobalChainsPanel />, <CityPanel />, <DatePickerPanel/>, and <SearchCTA />	
useReducer	N/A		
useContext	N/A		
URL/Router	N/A		
React Query Cache	[‘brands’] -> full brand list [‘brands’, ‘brandID’, ‘cities’] -> cities for active brand	<BrandList />	All GNB will follow same flow. Fetch from CDN, cache for TTL of 10min
Zustand/Redux	N/A		The JD mandates a Sustand state, but important to note it is

			not necessarily needed in page 1

4. Interface — API Contracts

Proposed contract shape for the backend team, based on the interactions the UI/UX guide requires. To be confirmed once real endpoints are available.

GET /api/brands

Request	**Response**
Headers: none required (public ref data) Payload: N/A Query params: none	2xx → 200 OK { brands: Brand[] } Type Brand = { id: uuid, name: string, logoURL: string, etc. } 4xx → None found 5xx → 500 brand data unavailable; <BrandList /> should render last cached list if present, otherwise an inline error state – must not block mega menu from opening

GET /api/brands/:brandID/cities

Request	**Response**
Headers: none required (public ref data) Payload: N/A Query params: brandID:string	2xx → 200 OK { cities: City[] } Type City = { id: uuid, name: string, countryCode: string, propertyCount: number} 4xx → 404 – unknown brandID. <CityPanel/> shows “no cities available” instead of error 5xx → 500 city data unavailable; <CityPanel /> should render last cached list if present, otherwise an inline error state – must not block mega menu from opening

GET @/api/availability

Request	**Response**
Headers: none required (public ref data) Payload: N/A Query params: brandID:string, city:string, checkIn:number, checkOut:number	2xx → 200 OK { cities: City[] } Type City = { isAvailable: boolean, resultsURL: “/results?brandID={brandID}&cityID={cityID}&checkIn:{checkIn}&checkOut={checkOut}”}

	4xx → 404 – unknown brandID. <CityPanel/> shows “no cities available” instead of error 5xx → 500 city data unavailable; <CityPanel /> should render last cached list if present, otherwise an inline error state – must not block mega menu from opening

Used by the Search CTA to validate before routing to the results page — exact response shape TBD with backend team; this is a starting proposal.

5. Optimizations

5.1 Performance

- Media, Copy, Shell: stored in CDN for fast lookup and retrieval cache to remove server load
- Images: next/image with responsive srcset; hero media served at viewport-appropriate resolution, lazy-loaded below the fold.
- BrandList virtualization (react-window) — with 110+ DOM nodes otherwise mounted on every mega-menu open.
- Code-split the MegaMenu and DatePicker bundles — not needed on initial paint, loaded on first hover/interaction.
- Route-level prefetching: prefetch /brands as soon as the user hovers the Global Chains nav item, before the menu finishes opening.
- CSS transitions preferred over JS-driven animation where possible (transform/opacity only, to stay on the compositor thread).
- Loading skeletons to prevent layout shift
- Debounce + AbortController for typing in search for about 500ms before showing search results to prevent multiple reqests per keystroke

5.2 Observability

- “Pending” → “success” as p50/p95 canary for error health to measure latency for loading times for loading brands, and cdn media.
- “Pending” → “success” as p50/p95 canary for error health to measure latency for debounce results
- “Failed” metrics categorized by error reason to detect system degradation
- Web Vitals (LCP, INP, CLS) tracked from first deploy — hero image is the likely LCP element and worth watching closely.
- Basic error boundary around MegaMenu / DatePicker so a data-layer failure degrades gracefully instead of breaking navigation.
- Full analytics/observability tooling (Sentry, RUM, etc.) treated as a Phase 1 stretch item, not a Milestone 1 blocker.

6. Open Questions for Client

- Expected behavior on mobile for the mega-menu (full-screen takeover vs. accordion)?